

Data Structures – Unit 03

Bucharest University of Economic Studies

Memory classes, Bit structures and operators, User data types



Memory classes

Define specific types of variables in order to differentiate between:

- allocation time;
- the type of memory that they use;
- the lifespan of a variable;
- the area of visibility, called the scope;



Declarations and Types

- Type specifiers:
 - void, char, short, int, long, float, double, signed, unsigned, struct/union, enum
- Type qualifiers:
 - const and volatile
- Storage-class specifier:
 - auto, register, static, extern, typedef, __declspec



Storage class specifiers

Specifier	Storage	Initial value	Scope	Lifespan
auto	Stack Segment	Garbage	Inside block	End of block
extern	Data Segment	Zero	Global	Entire runtime
static	Data Segment	Zero	Inside block	Entire runtime
register	CPU Registers	Garbage	Inside block	End of block



Memory classes

Automatic class variables are defined using the **auto** keyword or implicitly if nothing is specified:

```
{  
    ...  
    auto a = 2;           //explicit  
    int b = 20;           //implicit  
    ...  
}
```



Memory classes

Automatic class

- The variables *are NOT* initialized with implicit values
- They must have an initializer
- They are automatically updated when entering the scope in which they are defined
- They are visible only in the scope where they are defined
- By exiting the visibility area, the memory assigned to them is freed
- They have local lifetime



Memory classes

Static class variables are defined using the **static** keyword or implicitly for a certain category of variables like global because they are stored in the data segment:

```
int b;  
...  
static int a; // initialized implicitly with 0  
...
```



Memory classes

Static class

- Variables are initialized with 0 or an explicit value
- 2 types of such variables: *internals*, defined in the body of a function, *externals*, defined outside any function, in the global area
- They are stored in a *permanent* program area called data segment
- They keep their values at different calls of a function in which they were defined
- They have global lifetimes



Memory classes

External class variables are used in a different area than the one in which they were defined. They are introduced, not defined, with the **extern** keyword:

```
int a = 5;           // definition – memory allocation
extern int b;        // declaration – compiler notification
{
    ...
    int a = 10;
    int b = 20;
    printf("Value for local b: %d, and for external b: %d", b, ::b);
    ...
}
```



Memory classes

Extern class

- They are global variables defined in separate compiled files
- This type of variables has two important moments: *definition* and *declaration*
- They are automatically initialized by compiler and have static meaning
- Definition is the stage in which the compiler allocates memory
- Declaration is the stage needed for using the variable after definition by preceding it with the *extern*



Memory classes

Register variables are defined using the **register** keyword and forces the allocation in one of the processor's registers:

```
{  
    ...  
    for(register int b = 0; b<100; b++);  
    ...  
}
```



Memory classes

Register class

- They have similar behavior as automatic variables
- Are stored in one of the processor's registers in limited number
- The register specifier is only allowed for int, char and pointer data types
- They can not be dereferenced
- They have local lifetimes



User defined types

Article

- Groups different types of primitive variables
- Aggregated type which uses compact memory

Bit structure

- A particular type of article which allows users access at bit level

Union

- Allows that the same memory region be defined as two or multiple different types of variables

Enumeration

- Is a list of constant integer variables with given names

Synonyms

- Are defined by using the typedef specifier



User defined types

The article is a collection of heterogeneous data stored in a compact region of memory. It allows users to bring together multiple variables in a single defined type entity.

```
struct name {  
    type  member_name;  
};
```



User defined types

Data padding for the article:

```
struct name {  
    type length;  
    type padding[];  
};
```



User defined types

- Bits structures allow bit access level;
- To access data at bit level programmers must:
 - define a structure which maps perfectly on the data which must be accessed;
 - load the pointer to the bit structure with the address of the respective data;
 - processing the bits without altering the meaning and internal representation of data;



User defined types

- Bit fields:

```
struct name {  
    type  field_name  : width;  
    int   : width;  
    ...  
    int   field_name  : width;  
};
```



User defined types

- The declared bit field has the following properties:
 - The type may be int, signed or unsigned;
 - The number of bits that a member can refer is specified by width;
 - If member name is missing, the number of bits specified in the width field can not be referred and accessed;
 - The fields are accessed the same way as in any other structure;
 - The address of fields can't be used;



Bitwise operators

&

- AND

|

- OR

^

- OR exclusive - XOR

~

- One complement (flips bit value)

>>

- Right shifting

<<

- Left shifting



Bitwise operators on a byte

signed char x	x after operation	value of x
x=7	0000 0111	7
x=x<<1	0 0000 111 0	14
x=x<<3	000 0111 0 000	112
x=x<<2	0 1 1100 00 00	-64
x=x>>1	1 110 0000 0	-32
x=x>>2	1 111 1000 00	-8



Bitwise operators

& (AND)	0	1
0	0	0
1	0	1

 (OR)	0	1
0	0	1
1	1	1

^ (XOR)	0	1
0	0	1
1	1	0



Bitwise operators

$$\begin{array}{r} 11000001 \\ \& 01111111 \\ \hline 01000001 \end{array}$$

AND

$$\begin{array}{r} 01111111 \\ \wedge 01111000 \\ \hline 00000111 \end{array}$$

XOR

$$\begin{array}{r} 10000000 \\ | 00000011 \\ \hline 10000011 \end{array}$$

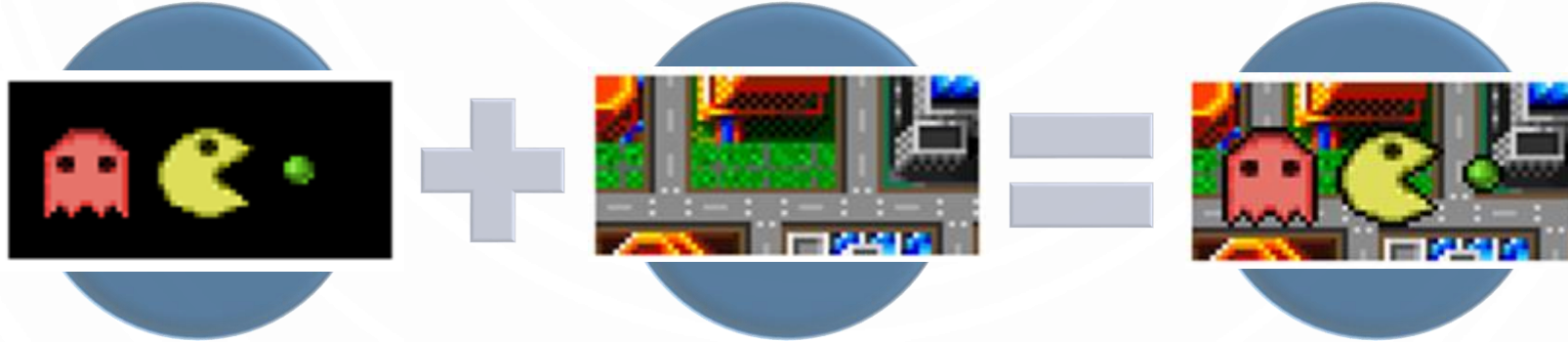
OR

$$\begin{array}{r} 00101100 \\ \sim 11010011 \\ \sim 00101100 \end{array}$$

NOT

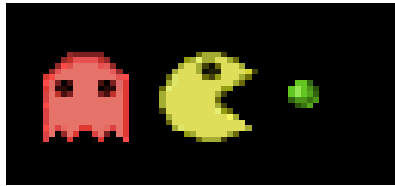


Bitwise operators

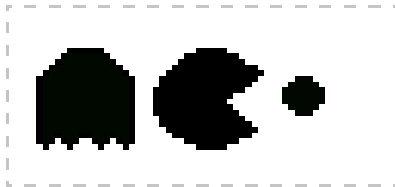


Bitwise operators

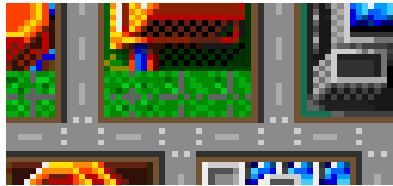
1.



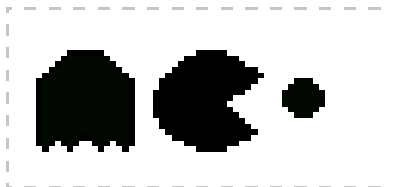
XOR



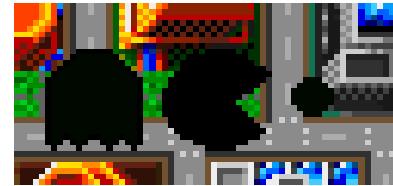
2.



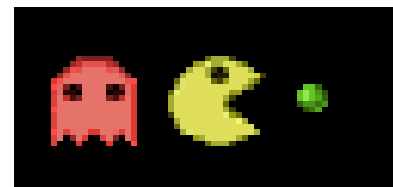
AND



3.



OR



User defined types

- Union defines an area of memory which at different execution moments can refer to different objects of different types and lengths;
- Is a composed data type of different variables that are mutually exclusive / mutual exclusion (mutex)



User defined types

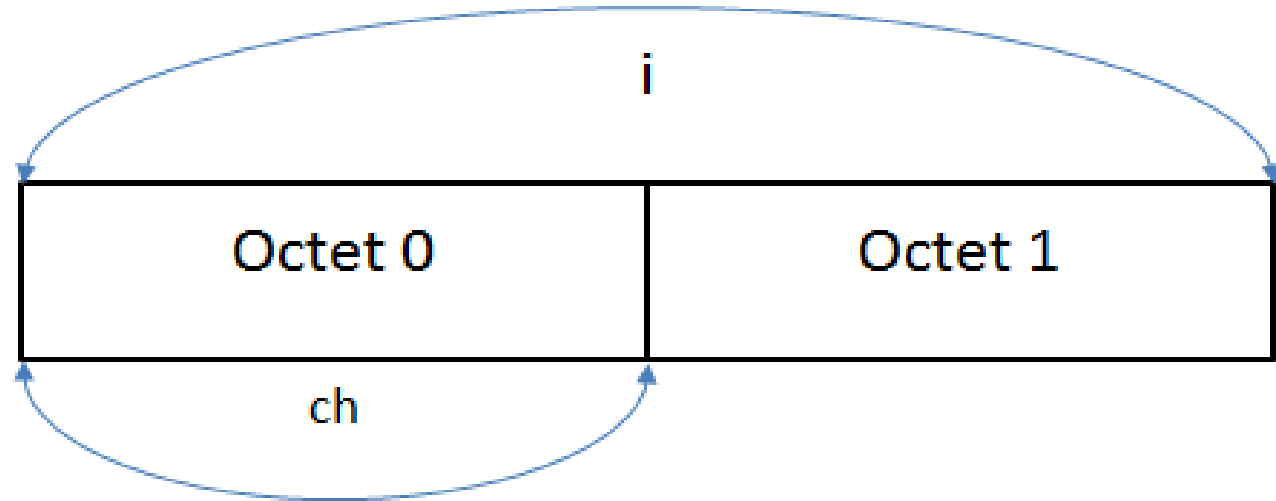
All the fields share the same memory region of size equal to the size of the largest member in the structure.

```
union nome_generic
{
    tip    nome_variabila_1;
    tip    nome_variabila_2;
    ...
    tip    nome_variabila_N;
};
```



User defined types

```
union tip_uniune  
{  
    short int i;  
    char  ch;  
};
```



User defined types

```
struct WORDREGS {
```

```
    unsigned ax, bx, cx, dx;
```

```
    unsigned si, di;
```

```
    unsigned cflag;
```

```
    unsigned flags;
```

```
};
```

```
struct BYTEREGS {
```

```
    unsigned char al, ah;
```

```
    unsigned char bl, bh;
```

```
    unsigned char cl, ch;
```

```
    unsigned char dl, dh;
```

```
};
```

```
union REGS {
```

```
    struct BYTEREGS    h;
```

```
    struct WORDREGS    x;
```

```
};
```



User defined types

- An enumeration is a set of integer type constants which specifies all the possible values of such a variable.

```
enum name  
{  
    enumerator [= initializer];  
} variable_list;
```



User defined types

The use of **typedef** keyword defines a synonym for an existing data type;

```
typedef tip new_name;
```

whereas **#define** statement can be used to define aliases for values as well;

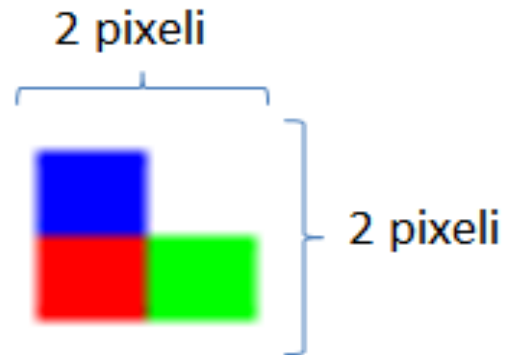
```
#define TRUE 1
```

```
#define FALSE 0
```



Bitmap structure

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F			
00000000	42	14D	46	00	2	00	00	00	00	3	00	00	36	00	4	00	00	28	00
00000010	00	00	02	00	00	00	02	00	00	00	00	01	00	18	00	00	00	00	00
00000020	00	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000030	00	00	00	00	00	00	00	00	5	FF	00	6	FF	00	7	00	FF	00	8
00000040	00	FF	9	FF	00	10	00												

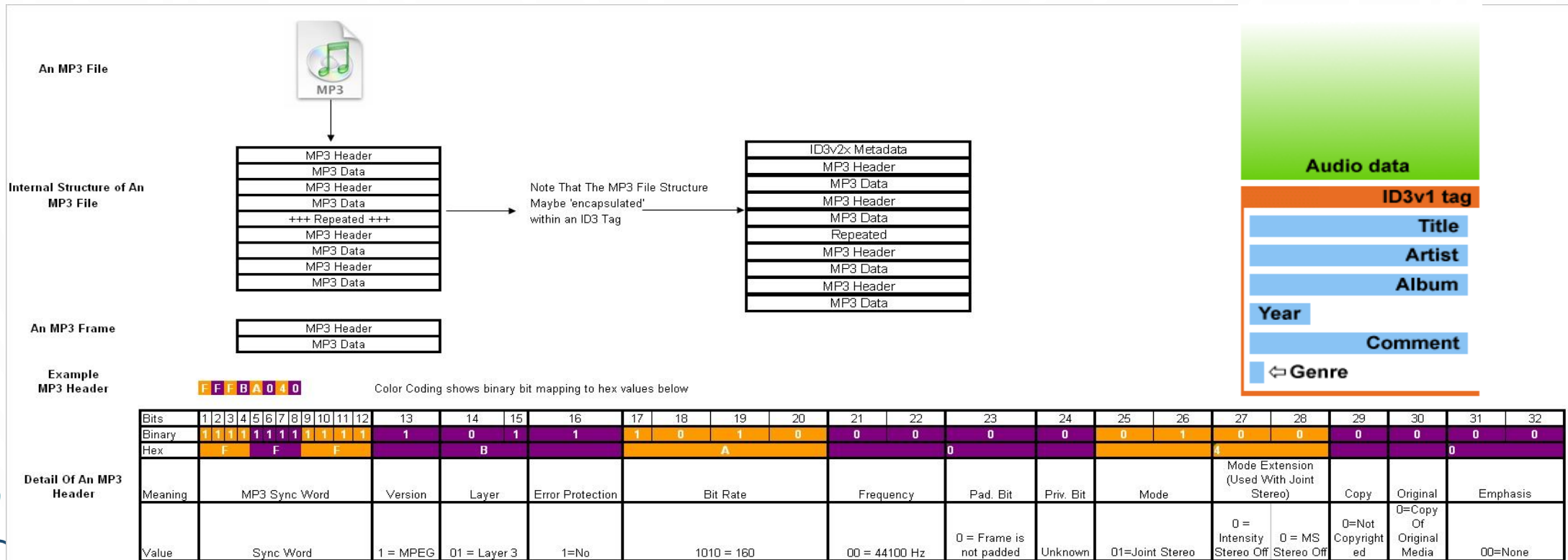


Bitmap structure

Structure		Legend
Bitmap Header		14 Bytes area
	1	4B, file signature, <i>4D42h</i>
	2	4B, file size, <i>00000046h</i>
	3	4B, reserved area, <i>00000000h</i>
	4	4B, image data offset, <i>00000036h</i>
Bitmap InfoHeader		40 Bytes area
Bitmap Data (pixels)		Pixels stored in RGB format / 1B per color
	5	3B pixel – Red color, <i>FF0000h</i>
	6	3B pixel – Green color, <i>00FF00h</i>
	7	2B padding after full line
	8	3B pixel – Blue color, <i>0000FFh</i>
	9	3B pixel – White color, <i>FFFFFFh</i>
	10	2B padding after full line



MP3 structure



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

